

CSE spring 1st 23 FINAL QUISTION SOLUTION



MADE BY SAHARIYAR ISLAM TANVIR (CSE 57 1AM C241023)

1 a answer)

```
cccccccccccccccc=15
```

1 b answer)

```
#include<stdio.h>

int main(){
    int n,i,j,x;
    scanf("%d",&n);
    if(n%2==0){ <---- //important line
        x=(n+5);
    }else{
        x=(n+4);
    }
    for(i=1;i<=n;i++){
        for(j=1;j<=x;j++){
            printf("*");
        }
        x-=2;
        printf("\n");
    }
    return 0;
}
```

1 c answer)

----->

```
#include<stdio.h>

int main(){

    int i,j,sum=0,n,cnt=0,x;

    double avg=0;

    scanf("%d",&n);

    for(i=1;i<=n;i++){

        scanf("%d",&x);

        if(x<0){

            break;

        }

        if(x%5==0 || x%7==0){

            continue;

        }else{

            cnt++;

            sum+=x;

        }

    }

    avg=(sum/cnt);

    printf("sum= %d\n average=%.2lf",sum,avg);

    return 0;

}
```

1 c answer or)

----->

```

#include<stdio.h>

int main(){

    int i,j,n,x;

    scanf("%d",&x);

    for(i=1;i<=x;i++){

        scanf("%d",&n);

        if(n<=1){

            printf("not a prime number\n");

        }

        while(n<=1){

            if(n==2){

                printf("prime number\n");

            }else if(n%2==0){

                printf("not a prime number\n");

            }else{

                for(i=3;i<=sqrt(n);i+=2){

                    if(n%i==0){

                        printf("Not a prime number\n");

                    }else{

                        printf(" prime number\n");

                    }

                }

            }

            break;

        }

    }

    return 0;

```

}

2 a answer)

----->

i) 1 2 3 4 5

ii)

```
#include<stdio.h>

int magic(int n){

    int j,sum=0;
    for(j=1;j<=n;j++){
        sum+=j;
    }
    return sum;
}

int main(){
    int i;
    for( i=1;i<=5;i++){
        printf("%d ",magic(i));
    }
    return 0;
}
```

2 b answer)

----->

same as 2 c or aut 23

2 b or --->

()

(2 c answer as n=53)

----->

53 26 13 6 3 1 1 3 6 13 26 53

3 a answer)

----->

```
#include<stdio.h>
```

```
int main(){
```

```
    int i ,arr[10]={0,0,0,1,4,7,10,0,0,0};
```

```
    for(i=9;i>=0;i--){
```

```
        printf("%d ",arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

3 b answer)

----->

1) 1 0 0 0 0

2) 1 1 2 3 4 (most important revised again)

3 c answer)

----->

```
#include<stdio.h>

int main(){

    int i,j,n,f=0;
    scanf("%d",&n);
    int arr[n];
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(i=0,j=n-1;i<j;i++,j--){
        if(arr[i]%2==0 && arr[j]%2==0 || arr[i]%2!=0 && arr[j]%2!=0){
            f=1;
        }else{
            f=0;
        }
    }
    if(f==1){
        printf("yes its qualindrom\n");
    }else{
        printf("yes its not qualindrom\n");
    }

    return 0;
```

```
}
```

3 c or answer)

----->

same as 3 c aut 23 (palinderom)

4 a answer)

----->

```
#include<stdio.h>
```

```
int main(){
```

```
    int i,j,row,col,s,cnt=0,sum=0;
```

```
    int arr[100][100];
```

```
    scanf("%d%d%d",&row,&col,&s);
```

```
    for(i=0;i<row;i++){
```

```
        for(j=0;j<col;j++){
```

```
            scanf("%d",&arr[i][j]);
```

```
        }
```

```
    }
```

```
    for(i=0;i<col;i++){
```

```
        sum=0;
```

```
        for(j=0;j<row;j++){
```

```
            sum+=arr[j][i];
```

```

    if(sum==10){
        cnt++;
    }
}
}

printf("%d",cnt);

return 0;
}

```

4 b answer)

----->

cseiiuc

7

cseiiuc

cseiiuc

4 c or answet)

----->

```

#include<stdio.h>

```

```

#include<string.h>

```

```

int main(){

```

```

    char str[100];

```

```

    int i,j,cnt=0;

```

```

    gets(str);

```

```

    for(i=0;str[i]!=0;i++){

```

```

        if(str[i]=='a' || str[i]=='A' || str[i]=='e' || str[i]=='E' ||

```



```

str[i]=='i' || str[i]=='l' || str[i]=='o' || str[i]=='O' ||
str[i]=='u' || str[i]=='U'){
    cnt++;
}
}
printf("number of vowel %d",cnt);
return 0;
}

```

5 a answer)

----->

A pointer is a variable that stores the address of another variable. Unlike other variables that hold values of a certain type,

pointer holds the address of a variable. For example, an integer variable holds (or you can say stores) an integer value,

however an integer pointer holds the address of a integer variable.

The address-of operator is used to obtain the memory address of a variable, while the dereference operator is used to access the value

stored at a memory address pointed to by a pointer.

5 a or answer)

----->

Bitwise operators in C are used to perform bit-level operations on integer data types. These operators work directly on the individual

bits of the binary representation of the numbers.

The bitwise AND operator (&) compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1,

the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0

The bitwise OR (|) operator returns a number or BigInt whose binary representation has a 1 in each bit position for which the corresponding

bits of either or both operands are 1

(5 b same as 5b aut 23)

**5 c answer ---->*

```
#include <stdio.h>

int main()
{
    int i,arr[100],n;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        scanf("%d",&arr[i]);
    }
    FILE *fr = fopen("cse1122.txt", "w");
    fprintf(fr, "%d\n", n);
    for (i = 0; i < n; i++)
    {
        fprintf(fr, "%d\n", arr[i]);
    }
    fclose(fr);
}
```

```
fr = fopen("cse1122.txt", "r");
fscanf(fr, "%d", &n);
for (i = 0; i < n; i++)
{
    fscanf(fr, "%d", &arr[i]);
}
fclose(fr);

printf("%d\n", n);
for (i = 0; i < n; i++)
{
    printf("%d\n", arr[i]);
}

return 0;
}
```